

Chapter 1

Cost, Memory & Power Estimation

ENEL712 Embedded Systems Design — Lecture 13

Summary

Estimating costs, memory, and power for embedded products: manufacturing economics, RAM/Flash budgets and risks, and battery-life estimation with sleep modes.

Topics Covered

- Cost estimation
- Memory estimation (Flash, RAM, EEPROM)
- Power estimation methods
- Power-saving techniques

1.1 Cost Estimation

- Manufacturing costs = components + PCB fab + assembly + testing + distribution.
- Component selection (e.g., LDO): balance price, availability, performance (dropout, current), and supply risk.
- Economies of scale: a regulator at \$0.509 per unit can fall to \$0.125 at >20,000 units.
- Sustainability and RoHS: pick RoHS-compliant components — limits on lead, mercury, etc.
- PCB fab/assembly: local (e.g., Circuit Labs, PCB Zone) vs cheaper overseas; full-service options (e.g., Triode) cover programming and testing.

1.2 Memory Estimation

1.2.1 C program storage and overhead

- Even an empty while(1) consumes Flash (e.g., 176 bytes) due to initialisation code and interrupt vectors.
- Local variables in small programs may live in GPRs to save RAM; globals always live in RAM.

- Initialised globals require space in BOTH Flash and RAM — Flash holds the constant, the startup code copies it into RAM. For estimation, assume initialised vars consume double memory.

1.2.2 Estimation challenges and risks

- Flash usage depends on code structure, language, libraries, optimisation flags, and floating point.
- Avoid recursion: rapidly consumes stack and is invisible to static memory estimates — risk of unhandled overflow.

1.3 Power Estimation

1.3.1 Constant loads

Lifetime = battery capacity (mAh) $\div$ total current (mA). Capacity rating depends on C-rate, age, cycles, and cutoff voltage.

1.3.2 Varying loads

- Rough method: schedule of operating modes per hour; integrate current over each state's duration.
- Accurate method: measure current over time and numerically integrate.

1.4 Power-Saving Techniques

- Power scales with supply voltage and clock frequency — slower and lower means less.
- Sleep modes (Idle, Power-down, etc.) shut down unused internal modules.
- Power Reduction Registers (PRR) gate the clock to specific peripherals.
- Duty cycling — minimise time in active state, maximise time in sleep — is the dominant lever for battery life.

1.5 Use Cases & Applications

- Costing a product variant: redoing the BOM with cheaper alternates and re-running the current tally tells you how much margin you can shave.
- Memory budgeting before code: catches projects that won't fit on the chosen MCU before sinking weeks into firmware.
- Battery sizing: the same energy estimation underpins every IoT sensor, wearable, and remote control.

1.6 Formulas & Calculations

1.6.1 Battery life (constant load)

$T_hours \approx Capacity_mAh / I_mA$ (true only at the rated C-rate and temperature).

1.6.2 Battery life (duty-cycled load)

$I_{avg} = D \cdot I_{active} + (1 - D) \cdot I_{sleep}$, where $D = t_{active} / (t_{active} + t_{sleep})$. Then $T_{hours} \approx \text{Capacity_mAh} / I_{avg_mA}$.

1.6.3 Memory rule of thumb

Initialised global variables eat both Flash and RAM. For estimation, treat them as costing double until the linker map says otherwise.

1.6.4 Volume cost reduction

Component price typically follows 'price $\propto 1 / \text{volume}^k$ ' with $k \approx 0.2\text{--}0.4$ across normal cut tape vs reel breakpoints. A $1 \rightarrow 1,000$ jump can drop unit price $3\text{--}5\times$.

1.7 Worked Example: Sensor Node Battery Life

Spec: 1-year service life, AA battery ($\sim 2,000$ mAh).

- Required $I_{avg} \leq 2,000 \text{ mAh} / (365 \times 24 \text{ h}) \approx 0.228 \text{ mA} = 228 \mu\text{A}$.
- If $I_{active} = 50 \text{ mA}$ (radio TX) and $I_{sleep} = 4 \mu\text{A}$, allowable $D \leq (228 - 4) / (50,000 - 4) \approx 0.0045 = 0.45 \%$.
- On a 60-second wake cadence, $t_{active} \leq 0.27 \text{ s}$ — pushes you to fast-startup radios and pre-computed payloads.

1.8 Memory Budget Worksheet

- Estimate Flash: empty firmware + libraries + features. Add 20 % margin for late changes.
- Estimate RAM: globals + biggest realistic stack depth + buffers (UART/SPI). Forbid the heap to keep this deterministic.
- If Flash is tight, audit floating-point usage — `printf("%f")` drags in the entire fp library.
- If RAM is tight, move large constants into Flash with `PROGMEM/const`.

Key Definitions

RoHS

Restriction of Hazardous Substances directive limiting materials like lead and mercury.

Program Memory (Flash)

Non-volatile firmware storage; persists across power cycles.

Data Memory (RAM)

Volatile memory for runtime variables, stack, and buffers.

EEPROM

Byte-addressable non-volatile memory for calibration/settings.

Recursion

A function calling itself; risky in embedded systems due to stack consumption.

Duty Cycling

Toggling between high-current active and low-current sleep states to save energy.

Power Reduction Registers (PRR)

Registers that disable the clock to specific peripherals.

LDO Regulator

Linear regulator with very small required V_{in} – V_{out} dropout.

Economies of Scale

Per-unit cost reduction as production volume rises.

GPRs

General Purpose Registers; fast CPU storage that can hold local variables.

BOM (Bill of Materials)

The complete list of components in a product, with quantities and per-unit costs.

C-Rate

Charge/discharge rate expressed as a multiple of capacity per hour ($1C$ = full discharge in 1 h).

Linker Map

File produced by the linker showing exactly how much Flash and RAM each module/symbol consumes.

Duty Cycle (power)

Fraction of time spent in the high-current active state; the dominant lever for battery life.

Sources

- Lecture 13.pdf